

Fitness Function Sensitivity in Search-Based Puzzle Level Generation

Hans Schaa

*Doctoral Program in Engineering Systems
Universidad de Talca
Curicó, Chile
hschaa12@alumnos.utalca.cl*

Nicolas A. Barriga*

*Faculty of Engineering
Universidad de Talca
Talca, Chile
nbarriga@utalca.cl*

Abstract—TODO: write abstract here.

Index Terms—Sokoban, difficulty generation, metaheuristics, procedural content generation, objective function sensitivity

I. INTRODUCTION

Solving difficulty is a critical component of puzzle level design that directly shapes the player experience. Designing puzzle content that is both varied and appropriately challenging is a demanding task, since crafting each instance by hand does not scale well, and ensuring that a given puzzle is solvable, non-trivial, and enjoyable requires careful, time-costly design [1]. This sensitivity is not unique to any single puzzle type: in sliding-block puzzles, for instance, small variations in a level’s structure, such as the size of the movable pieces, can drastically alter the size of the search tree and the effort required to find a solution [2]. Sokoban, the puzzle subgenre studied in this work, exhibits this same sensitivity and traces back to the classic game created in 1981 by Hiroyuki Imabayashi [1]. The complexity of solving Sokoban levels is PSPACE-complete [3], making the accurate estimation of level difficulty a considerable challenge.

Procedural Content Generation (PCG) has emerged as an effective solution for designing large volumes of levels, providing a controlled framework for generating content with specific metrics for solvability, difficulty, and diversity. To this end, a variety of approaches based on computational algorithms have been developed [4], among which the following stand out:

Constructive Algorithms that generate content according to predefined rules, typically operating under a one-shot execution scheme (noise-based methods, rule-based systems, grammars, etc.).

Search-Based These approaches explore a solution space defined by the **level’s representation**, which is then guided and optimized through fitness functions via various metaheuristics (evolutionary algorithms, quality-diversity algorithms, etc.).

This research was partially funded by the National Agency for Research and Development (Agencia Nacional de Investigación y Desarrollo, ANID Chile) through grant ANID-Subdirección de Capital Humano/Doctorado Nacional/2023-21230824, and by the ANID Sub-directorate of Applied Research (Subdirección de Investigación Aplicada, SIA) through grant IT24I0080.

* Corresponding author

Machine Learning These methods treat content generation as a function-**approximation** problem, where the system optimizes its parameters to map inputs to levels based on patterns extracted from training data (neural networks, probabilistic models, constraint-based methods, sequential models, among others).

In search-based generation, the fitness function determines the quality of the solutions found. Its design is critical in modern generative systems, as it is the key factor that ensures the resulting levels satisfy both system requirements and player expectations. However, the literature reveals a significant gap in this regard. There is only limited consensus within the community on how to rigorously evaluate these systems [5], and recent studies warn that the uncritical use of fitness functions may provide a superficial or even misleading view of the true quality of the generated content [5]. Furthermore, research has historically prioritized the development of novel generative algorithms while neglecting the justification of system configuration decisions [6]. Together, these issues highlight the need for a deeper analysis of generative processes, placing greater emphasis on the overall quality of the results rather than on the mere production of large quantities of content.

To study the interplay between metaheuristics and difficulty metrics in search-based level generation, this paper compares three algorithms across three objective functions for generating initial box, goal, and player configurations on Sokoban boards. The research questions addressed are the following:

RQ1 Which metaheuristic achieves the best aggregate performance across the suite of objective functions, and to what extent is this hierarchy landscape-dependent?

RQ2 Which metaheuristic demonstrates the highest robustness to the choice of objective function, minimizing intra-algorithm variance across disparate fitness landscapes?

The findings of this work have direct implications for designers of search-based generative systems. Understanding which metaheuristics are more robust to changes in the fitness function enables more informed decision-making during the early stages of game development. Likewise, identifying which objective functions induce divergent behaviors across algorithms highlights that the choice of a difficulty proxy should not be treated as a secondary parameter, but rather as a central

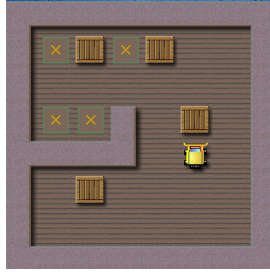


Figure 1: Sokoban board example.

architectural decision within the generative system.

In practice, these findings can reduce the cost of experimentation when configuring a procedural generator. Instead of exhaustively testing all possible combinations of algorithms and evaluation metrics, designers can rely on the sensitivity patterns documented in this study to narrow the decision space from the outset.

The remainder of this document is organized as follows. Section II presents the theoretical background and related work. Section III describes the problem formulation, including its representation, decision variables, objective function, and constraints. Section IV details the methodology designed to address the research questions. Section V presents the experimental design and the results obtained. Finally, Sections VI and VII discuss the findings and present the conclusions along with directions for future work.

II. BACKGROUND

Sokoban-type puzzles are typically presented as a grid in which the player must move objects to reach a goal state. Figure 1 shows a Sokoban board where the player, goals, boxes, and walls that restrict movement can be identified. The core gameplay mechanic consists of pushing boxes until they are placed on their corresponding goal tiles. A puzzle is considered solved when all boxes are positioned on the goals. The example illustrated in Figure 2 demonstrates the basic gameplay dynamics, in this case, the player moves one step to the right and then one step upward to complete the puzzle.

Sokoban can exhibit deadlock states, that is, game configurations from which the player can no longer complete the puzzle, regardless of the moves performed [7]. Figure 3 illustrates an example of such a state. The area highlighted in red represents a region where a deadlock occurs if a box is pushed into it. Under these circumstances, the player cannot remove the box, since the game’s mechanics allow boxes to be pushed but not pulled. The presence of deadlocks, together with the combinatorial explosion of the state space and the PSPACE-complete nature of the Sokoban solving problem, makes the automatic generation of game boards a particularly challenging task.

A. Procedural Content Generation for Sokoban

Procedural generation of Sokoban levels has primarily been approached through two of the three paradigms described

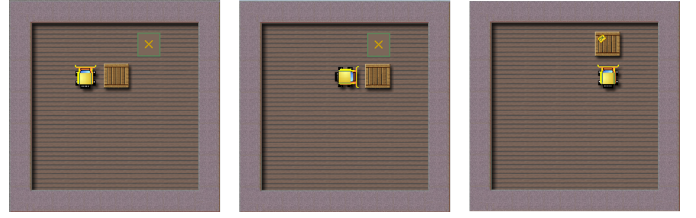


Figure 2: Steps taken by the player to solve the board.

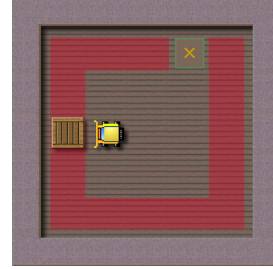


Figure 3: Toy example illustrating a *deadlock*. The red area indicates the region in which a *deadlock* occurs if a box is pushed into it.

above: Search-Based Procedural Content Generation and Machine Learning. For example, in the search-based approach, [8] uses evolutionary algorithms to generate boards under the assumption that levels requiring a greater number of box pushes to be completed will exhibit greater perceived difficulty when solved.

In practice, it is often desirable to generate game levels that simultaneously satisfy multiple characteristics. Consequently, multi-objective optimization algorithms have been used not only to optimize difficulty, but also to balance several potentially conflicting criteria. In this context, the NSGA-II algorithm has been employed to evolve level generators for games such as Sokoban and Zelda, simultaneously optimizing attributes such as the number of boxes, the absolute difference with respect to goals, and solution length [9]. Similarly, the Two_Arch2 algorithm has been used to balance aesthetic and structural objectives in Sokoban, including spatial diversity and the amount of space in generated boards [10]. Beyond single-objective search, Quality-Diversity (QD) algorithms extend evolutionary principles by simultaneously optimizing for both quality and behavioral diversity. One of the canonical QD algorithms in the PCG literature is MAP-Elites [11]. In particular, an extension called Covariance Matrix Adaptation MAP-Elites (CMA-ME) has been applied to level generation for Sokoban, alongside mazes and Zelda, demonstrating that the algorithm can produce diverse yet capable generators that satisfy complex solvability criteria [12]. However, a systematic study of metaheuristic sensitivity to objective function choice in Sokoban generation remains, to the best of our knowledge, unexplored.

In contrast to approaches based exclusively on Machine Learning, particularly those founded on deep learning techniques such as training generative agents [13], metaheuristic-

based methods do not require large volumes of training data. This work is grounded on that premise and differentiates itself from the existing literature by conducting a comparative evaluation of three metaheuristics under three distinct fitness functions.

B. Difficulty estimation for Sokoban

Since the primary goal of many level generators is to maximize challenge without frustrating or boring the player, defining difficulty mathematically is a fundamental step. To this end, several metrics have been proposed to estimate it. For example, features derived from the behavior of the A* algorithm have been used, including the final size of the closed list [14], the effective branching factor, the number of deadlocks, and the number of repeated nodes, to assess the degree of redundancy present in a Sokoban level. These latter metrics were validated across three different puzzle types, including Safari Rush Hour and Asteroid Escape, supporting the hypothesis that they can serve as effective difficulty proxies [15]. In addition, computational models based on human problem-solving behavior, the presence of counterintuitive moves, and problem decomposition have been developed, showing promising results through their correlation with user-provided difficulty assessments [16].

III. PROBLEM FORMULATION

A. Representation

Due to the discrete nature and grid-based structure of Sokoban boards, they can be represented using matrices or vectors. In this work, a representation based on a one-dimensional integer vector is adopted, where each position corresponds to a cell of the board. Table I presents the mapping between the numerical values used and the game elements they represent.

Equation (1) defines a Sokoban board B as a one-dimensional vector of size $w \cdot h$, where w and h denote the width and height of the board, respectively. Equation (2) specifies the set of values that each board element may assume, while Equation (3) defines the domain of the index i , corresponding to the positions within the vector.

$$B = \{b_1, b_2, \dots, b_i\} \quad (1)$$

$$b_i \in \{0, 1, 2, 3, 4, 5, 6\} \quad (2)$$

$$1 \leq i \leq w \cdot h \quad (3)$$

B. Decision Variables

The decision variables correspond to all board positions that do not contain walls, since walls remain fixed throughout the optimization process. Consequently, the set of decision variables is defined as follows:

$$X = \{b_i \in B \mid b_i \neq 1\} \quad (4)$$

ID	Element Type
0	Floor.
1	Wall.
2	Player.
3	Box.
4	Goal.
5	Player over the goal.
6	Box over goal.

Table I: Numerical values used and their correspondence to the elements of the Sokoban puzzle.

Parameter	Description
h	Height of the Sokoban board.
w	Width of the Sokoban board.
B	One-dimensional vector representing the complete state of the board.
$maxBoxes$	Maximum number of boxes allowed on the board.

Table II: Mathematical model parameters.

Each decision variable ($b_i \in X$) may take any of the values defined in Table II, except the value corresponding to walls.

C. Fitness Functions

As discussed in the previous section, several studies have proposed different metrics for estimating difficulty. In this article, we employ three parameters derived from the behavior of the A* algorithm as fitness functions:

- 1) **Pushes:** The number of box pushes required to solve the level.
- 2) **Deadlocks:** The number of deadlocks encountered by the A* algorithm while searching for a solution.
- 3) **Repeated Nodes:** The number of duplicate states generated and subsequently discarded by the solver after verifying that they have already been visited.

Let $g(B)$ denote one of the objective functions described above. The optimization problem consists of maximizing this function, thereby promoting the generation of boards whose solutions require greater cognitive effort from the player.

D. Restrictions

To facilitate the definition of the constraints, the auxiliary variables $Total_{boxes}$, $Total_{goals}$, $Total_{players}$, and $Total_{empty}$ are introduced, representing the total number of boxes, goals, players, and empty spaces, respectively. These quantities are computed over the board B , considering all elements from $i = 1$ to $i = w \cdot h$. The auxiliary indicator variables $isBox$, $isGoal$, $isPlayer$, and $isEmpty$ are used to identify the type of element present at each position of the board.

Based on the previous expressions, the following constraints are defined:

$$maxBoxes = \max \left(3, \min \left(7, \left\lfloor \frac{Total_{empty}}{5} \right\rfloor \right) \right) \quad (9)$$

$$Total_{boxes} = \sum_{i=1}^{w \cdot h} isBox_i \quad (5)$$

$$Total_{goals} = \sum_{i=1}^{w \cdot h} isGoal_i \quad (6)$$

$$Total_{players} = \sum_{i=1}^{w \cdot h} isPlayer_i \quad (7)$$

$$Total_{empty} = \sum_{i=1}^{w \cdot h} isEmpty_i \quad (8)$$

$$isBox_i = \begin{cases} 1, & \text{if } b_i = 3 \vee b_i = 6 \\ 0, & \text{if } b_i \neq 3 \wedge b_i \neq 6 \end{cases}$$

$$isGoal_i = \begin{cases} 1, & \text{if } b_i = 4 \vee b_i = 5 \vee b_i = 6 \\ 0, & \text{if } b_i \neq 4 \wedge b_i \neq 5 \wedge b_i \neq 6 \end{cases}$$

$$isPlayer_i = \begin{cases} 1, & \text{if } b_i = 2 \vee b_i = 5 \\ 0, & \text{if } b_i \neq 2 \wedge b_i \neq 5 \end{cases}$$

$$isEmpty_i = \begin{cases} 1, & \text{if } b_i = 0 \\ 0, & \text{if } b_i \neq 0 \end{cases}$$

Figure 4: Element-count summations by category for board B .

$$Total_{boxes} \leq maxBoxes \quad (10)$$

$$Total_{boxes} = Total_{goals} \quad (11)$$

$$Total_{boxes} > 0 \quad (12)$$

$$Total_{players} = 1 \quad (13)$$

The constraints are described as follows:

- 1) Equation 9 computes the maximum number of boxes allowed on the board, $maxBoxes$, as a function of the number of empty cells $Total_{empty}$. The value is clamped between a lower bound of 3 and an upper bound of 7, dividing the available empty space by 5. This adapts the box limit to the board's size while preventing excessive growth of the search space, whose complexity increases exponentially with the number of boxes. Equation 10 then enforces this bound.
- 2) Equation 11 establishes that the number of boxes must be equal to the number of goals.
- 3) Equation 12 guarantees the presence of at least one box on the board.
- 4) Equation 13 enforces the existence of exactly one player.

These constraints ensure only the basic structural validity of the board. Solvability is verified subsequently through the execution of a *solver*.

IV. METHODOLOGY

To address the research questions, three metaheuristics were implemented in C++: Simulated Annealing, a $(\mu + \lambda)$ -Evolution Strategy, and a Genetic Algorithm. All three share

the same termination criteria: a maximum of 2,000 fitness evaluations or 200 consecutive generations without improvement. The following sections detail the logic common to all metaheuristics, including population initialization and the mutation operators applied.

A. Initialization

The metaheuristic receives a Sokoban board containing only walls. As shown in Algorithm 1, the initialization phase populates up to pop_size individuals by randomly placing a player, a box, and a goal on the board. If the resulting individual yields a fitness value below -10^8 , it is considered invalid and the process retries up to 50,000 times. Once a valid individual is found, it is added to the population. This procedure is applied independently to each metaheuristic evaluated.

Algorithm 1 Population initialization

```

1: for  $i = 1$  to  $pop\_size$  do
2:    $valid \leftarrow \text{false}$ ,  $attempts \leftarrow 0$ 
3:   while  $\neg valid$  and  $attempts < 50000$  do
4:     Copy shell into board; randomly place @, $, .
5:      $fit \leftarrow \text{EVALUATE}(\text{board})$ 
6:     if  $fit > -10^8$  then
7:       Add individual with fitness =  $fit$  to population
8:        $valid \leftarrow \text{true}$ 
9:     end if
10:     $attempts \leftarrow attempts + 1$ 
11:   end while
12:   if  $\neg valid$  then return failure
13:   end if
14: end for
```

B. Mutations

Each metaheuristic explores the solution space through three equiprobable mutations. The first, *AddMutation* (Algorithm 2), places a box and a goal at random empty cells. Lines 1–4 enforce the constraint defined in Equation 10, which bounds the number of boxes to prevent excessive complexity that could exceed the limits of the test environment. The second, *MoveMutation* (Algorithm 3), relocates a randomly selected player, box, or goal to a random empty cell. The third, *RemoveMutation* (Algorithm 4), removes one box–goal pair from the board, ensuring at least one pair remains.

C. Simulated Annealing

This single-solution metaheuristic explores the search space by occasionally accepting solutions with lower fitness than the current one, to escape local optima. This acceptance is governed by the probability $e^{\Delta f/T}$, which decreases as the temperature T cools down. Consequently, the algorithm emphasizes exploration in early stages when T is high and shifts toward exploitation as T approaches $T_{\min}$. The hyperparameters involved are the following:

Algorithm 2 Add mutation

```

1: free  $\leftarrow$  COUNTFREECELLS(board)
2: max_boxes  $\leftarrow$  max(3, min(7,  $\lfloor \text{free}/5 \rfloor$ ))
3: if COUNTBOXES(board)  $\geq$  max_boxes then return false
4: end if
5: empty  $\leftarrow$  list of all empty cells in board
6: if |empty|  $<$  2 then return false
7: end if
8: Pick random cell  $p_1$  from empty; place $ at  $p_1$ 
9: Pick random cell  $p_2$  from empty  $\setminus \{p_1\}$ ; place . at  $p_2$ 
   return true

```

Algorithm 3 Move mutation

```

1: pos  $\leftarrow$  GETPOSITIONS(board, target)
2: if pos =  $\emptyset$  then return false
3: end if
4:  $p_{\text{src}}$   $\leftarrow$  random element from pos
5:  $p_{\text{dst}}$   $\leftarrow$  GETRANDOMDESTINATION(board, target)
6: if  $p_{\text{dst}}$  is invalid or  $p_{\text{src}} = p_{\text{dst}}$  then return false
7: end if
8: Clear  $p_{\text{src}}$ : if source was on a goal cell, restore . ; otherwise place  $\square$ 
9: Place target at  $p_{\text{dst}}$ : if destination is a goal cell, use composite state (+ or *) return true

```

- 1) T_0 : initial temperature, controls the degree of exploration at the start.
- 2) $T_{\min}$: minimum temperature, used as an additional termination criterion.
- 3) coolingRate: geometric decay factor applied to T at each iteration, defined as $T \leftarrow T \cdot (1 - \text{coolingRate})$.

D. Evolutionary Strategy (ES)

The $(\mu + \lambda)$ -ES is a variant of the Evolution Strategy in which the next generation is selected from the combined pool of μ parents and λ offspring, retaining only the fittest μ individuals. As shown in Algorithm 5, the metaheuristic is governed by the following hyperparameters:

latex

Algorithm 4 Remove mutation

```

1: boxes  $\leftarrow$  all cells containing a box in board
2: goals  $\leftarrow$  all cells containing a goal in board
3: if |boxes|  $\leq$  1 or |goals|  $\leq$  1 then return false
4: end if
5: Pick random  $p_b$  from boxes; remove box at  $p_b$ , restoring . if it was a composite cell
6: goals  $\leftarrow$  goals  $\setminus \{p_b\}$ 
7: if goals =  $\emptyset$  then
8:   Rollback  $p_b$ ; return false
9: end if
10: Pick random  $p_g$  from goals; remove goal at  $p_g$ , restoring occupant if composite return true

```

- 1) μ : Subset of individuals that will give rise to new solutions in each iteration.
- 2) λ : Total number of individuals to be generated in each iteration.
- 3) Mutation rate: Probability of applying the mutation operator at each iteration.

Algorithm 5 $(\mu + \lambda)$ Evolution Strategy

```

1: EVALUATE(population); best  $\leftarrow$  arg max fitness
2: while evaluations  $<$  maxEvaluations and stagnation  $<$  stagnationLimit do
3:   offspring  $\leftarrow \emptyset$ 
4:   while |offspring|  $<$   $\lambda$  and attempts  $<$  maxAttempts do
5:     child  $\leftarrow$  random parent from population  $\triangleright$ 
     Inherits fitness implicitly
6:     needsEvaluation  $\leftarrow$  false
7:     if rand()  $\leq$  mutationRate then
8:       Apply random mutation to child
9:       if mutation failed then continue
10:    end if
11:    needsEvaluation  $\leftarrow$  true
12:  end if
13:  if needsEvaluation then
14:    child.fitness  $\leftarrow$  EVALUATE(child);
    evaluations += 1
15:  end if
16:  if child is valid then
17:    add to offspring; update best if improved
18:  end if
19: end while
20: population  $\leftarrow$  top  $\mu$  of population  $\cup$  offspring by fitness
21: if no improvement then stagnation += 1
22: else stagnation  $\leftarrow$  0
23: end if
24: end while return best

```

E. Genetic algorithm

The Genetic Algorithm uses tournament selection to choose parent pairs. As shown in Algorithm 6, the crossover operator exchanges *interesting regions* between parents: horizontal or vertical segments of width s containing at least one game element (player, box, or goal). If the resulting child is structurally invalid, a single repair attempt is made before discarding it. Both crossover and mutation are applied independently with probabilities p_c and p_m respectively, meaning a child may inherit directly from p_1 without crossover. The full evolutionary process is shown in Algorithm 7, which incorporates elitist replacement, guaranteeing the global best individual survives every generation, and a recovery mechanism that fills any population deficit with mutated clones of the best-known individual.

Algorithm 6 Board Crossover

```

1: child  $\leftarrow$  parent1
2: regions  $\leftarrow$  horizontal/vertical segments of length  $s$  in
   parent2 containing at least one game element and no walls
3: if regions =  $\emptyset$  then return false
4: end if
5: Copy a random region from parent2 into child at the same
   coordinates
6: if child is not structurally valid then apply REPAIR(child)
7: end if return child is structurally valid

```

Algorithm 7 Genetic Algorithm

```

1: Evaluate population; best  $\leftarrow$  arg max fitness
2: while evaluations < maxEvaluations and stagnation <
   stagnationLimit do
3:   while |offspring| < offspringSize and attempts <
     maxAttempts do
4:      $p_1, p_2 \leftarrow$  TOURNAMENT(population)
5:     child  $\leftarrow$  CROSSOVER( $p_1, p_2$ ) with prob.  $p_c$ , else
      $p_1$ ; if failed continue
6:     Apply random mutation with prob.  $p_m$ ; if failed
     continue
7:     child.fitness  $\leftarrow$  EVALUATE(child);
     evaluations += 1; add to offspring
8:   end while
9:   population  $\leftarrow$  {best}  $\cup$  top(offspring, popSize - 1); fill
     deficit with mutated clones of best
10:   Update best and stagnation counter
11: end while return best

```

V. EXPERIMENTS AND RESULTS

All experiments were conducted on a machine equipped with an AMD Ryzen 7 2700X processor (8 cores, 16 threads, and a base clock frequency of 3.7 GHz) and 32 GB of RAM. The experiments were performed on Ubuntu 20.04.6 LTS running through Windows Subsystem for Linux (WSL) on a Windows 10 operating system. The details of the experimental setup are described below.

A. Base Board Selection

The test instances for IRace and the comparative study were selected using the K-Means clustering algorithm available in Python’s scikit-learn library (`sklearn.cluster`). The process began with the collection of two datasets. The first, proposed by Jordi Domènech [17], contains approximately 450 boards with diverse characteristics. To complement this collection, the dataset used in [18] was incorporated, resulting in a total of 1,422 instances. After a filtering process to remove duplicate cases, the final subset consisted of 1,273 boards.

Each board underwent a preprocessing step in which all boxes, goals, and the player were removed, yielding boards that contained only walls. These simplified layouts are referred to as *Board Templates* (BTs). As shown in Table III, each BT was characterized using a set of structural properties. Applying

K-Means, the collection was partitioned into two disjoint subsets: 6 tuning instances for IRace and, from the remaining boards, 20 instances for the metaheuristic comparison study. This procedure ensured a representative sample covering the full expressive range of the dataset.

Metric	Description
<i>wall_density</i>	Proportion of wall cells relative to the total bounding box area.
<i>open_space_ratio</i>	Fraction of navigable cells over the total interior area.
<i>connectivity</i>	Number of cells in the largest connected navigable region.
<i>aspect_ratio</i>	Ratio of the board width to its height.
<i>dead_end_ratio</i>	Proportion of navigable cells with exactly one free neighbor.
<i>avg_symmetry</i>	Average of horizontal and vertical symmetry scores.
<i>num_interior_regions</i>	Number of isolated or unreachable interior regions.

Table III: Structural metrics for Sokoban shells.

B. Hyperparameter Tuning via Iterated Racing

Hyperparameter tuning was performed using the `irace` package [19], designed to automate the search for optimal algorithm configurations. The process was executed with a budget of 1000 evaluations, distributed across 8 parallel processing cores. Furthermore, to ensure the computational tractability of the tuning process against degenerate hyperparameter configurations (which may induce infinite loops during the structural validation phase of the evolutionary cycle), a hard execution limit of 20 minutes was imposed per independent evaluation.

Table IV details the explored ranges and the final selected configuration. This conservative bound is justified by the PSPACE-complete complexity of Sokoban [3] (Section I): iteratively solving instances within an evolutionary algorithm under tuning carries the risk of unmanageable growth in both time and memory.

1) *Solver A**: To guarantee the solvability of the generated boards, an A*-based solver¹ is employed, compiled with GCC/G++ 11.4.0 under the C++17 standard and the `-O3` optimization flag. From its execution, the fitness metrics are extracted: pushes, *deadlocks*, and repeated nodes. To prevent excessive memory consumption during exhaustive searches, the solver incorporates an early termination criterion that aborts execution when either the number of generated nodes exceeds 350,000 or the runtime exceeds 120 seconds. In such cases, the board is considered invalid and assigned a fitness value of -1×10^9 . This penalty allows discarding boards with topologies that present bottlenecks or intractable deadlocks for the algorithm, which artificially inflate the search tree and excessively slow down the evolutionary cycle.

Hyperparameter	Range	FO1	FO2	FO3
Evolutionary Strategy				
μ	[2, 15]	—	—	—
λ	[15, 30]	—	—	—
mutRate	[0.01, 1.00]	—	—	—
Genetic Algorithm				
offspring	[10, 50]	—	—	—
maxFailed	[10, 50]	—	—	—
mutRate	[0.01, 1.00]	—	—	—
crossRate	[0.01, 1.00]	—	—	—
Simulated Annealing				
initTemp	[10.0, 1000.0]	—	—	—
coolRate	[0.0001, 0.05]	—	—	—
maxFailed	[10, 100]	—	—	—

Table IV: Parameter space and final configuration obtained via irace for each objective function FO1 (Pushes), FO2 (Deadlocks), and FO3 (Repeated).

C. Experiment 1: Aggregate Performance and Hierarchy Stability

One of the objectives of this work is to determine which metaheuristic achieves the best overall performance and to evaluate the stability of this ranking across the proposed *fitness* functions. To this end, the best *fitness* obtained across $N=30$ independent runs per algorithm over the 20 BTs were recorded, yielding a total of 5400 runs. Figure ?? presents the dispersion of these results, a superior algorithm is distinguished by higher medians and lower interquartile overlap.

TODO: Results

D. Experiment 2: Algorithm Robustness and Landscape Sensitivity

To address RQ2, this experiment evaluates the sensitivity of each metaheuristic to changes in the objective function. Since the three metrics operate on different orders of magnitude, the distributions obtained in the previous experiment were normalized using Min-Max scaling to the interval [0,1] per BT. Figure ?? illustrates the intra-algorithm behavior. Visually, an algorithm is considered robust when its boxes remain compact and aligned with one another, indicating that its effectiveness does not depend on the objective function employed.

TODO: Results

VI. DISCUSSION

a) RQ1 — Which metaheuristic achieves the best aggregate performance across the suite of objective functions, and to what extent is this hierarchy landscape-dependent?: TODO.

b) RQ2 — Which metaheuristic demonstrates the highest robustness to the choice of objective function, minimizing intra-algorithm variance across disparate fitness landscapes?: TODO.

A. Limitations

a) Internal validity: TODO.

b) External validity: TODO.

VII. CONCLUSION

TODO.

VIII. DECLARATION OF THE USE OF AI TECHNOLOGIES IN THE WRITING PROCESS

The authors used ChatGPT-5 and Grammarly to improve the grammar and readability of this manuscript. After using these tools, the authors reviewed and edited the content as needed, taking full responsibility for the content.

REFERENCES

- [1] B. De Kegel and M. Haahr, "Procedural puzzle generation: A survey," *IEEE Transactions on Games*, vol. 12, no. 1, pp. 21–40, 2019.
- [2] R. A. Hearn and E. D. Demaine, "Pspace-completeness of sliding-block puzzles and other problems through the nondeterministic constraint logic model of computation," *Theoretical Computer Science*, vol. 343, no. 1-2, pp. 72–96, 2005.
- [3] J. Culberson, "Sokoban is pspace-complete," 1997.
- [4] M. Guzdial, S. Snodgrass, and A. J. Summerville, *Procedural content generation via machine learning*. Springer, 2025.
- [5] O. Withington, M. Cook, and L. Tokarchuk, "On the evaluation of procedural level generation systems," in *Proceedings of the 19th International Conference on the Foundations of Digital Games*, 2024, pp. 1–10.
- [6] J. A. del Solar-Zavala and N. A. Barriga, "Representations for the procedural content generation of puzzle game instances: A systematic literature review," *IEEE Access*, vol. 14, pp. 50 396–50 413, 2026.
- [7] T. Cazenave and N. Jouandeau, "Towards deadlock free sokoban," in *Board Game Studies XIIIth Colloquium*, 2010.
- [8] H. Schaa and N. A. Barriga, "Generating entertaining human-like sokoban initial states," in *2021 40th International Conference of the Chilean Computer Science Society (SCCC)*, 2021, pp. 1–6.
- [9] A. Khalifa and J. Togelius, "Multi-objective level generator generation with marahel," in *Proceedings of the 15th International Conference on the Foundations of Digital Games*, 2020, pp. 1–8.
- [10] Q. Zhang, Y. Li, Y. Lin, H. Wang, and J. Liu, "Interpreting multi-objective evolutionary algorithms via sokoban level generation," in *2024 IEEE Conference on Games (CoG)*, 2024, pp. 1–2.
- [11] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," *arXiv preprint arXiv:1504.04909*, 2015.
- [12] S. Earle, J. Snider, M. C. Fontaine, S. Nikolaidis, and J. Togelius, "Illuminating diverse neural cellular automata for level generation," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2021, pp. 68–76.
- [13] Y. Zakaria, M. Fayek, and M. Hadhoud, "Procedural level generation for sokoban via deep learning: An experimental study," *IEEE Transactions on Games*, vol. 15, no. 1, pp. 108–120, 2022.
- [14] E. Valenzuela, H. Schaa, N. A. Barriga, and G. Patow, "Using search algorithm statistics for assessing maze and puzzle difficulty," *Entertainment Computing*, vol. 53, p. 100925, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1875952125000059>
- [15] H. Schaa, S. Quiroz, and N. A. Barriga, "Modeling puzzle difficulty via a* search behavior, solution paths, and spatial metrics," ResearchGate, 2026, preprint. [Online]. Available: <https://doi.org/10.13140/RG.2.2.29160.84484>
- [16] P. Jarušek and R. Pelánek, "Difficulty rating of sokoban puzzle," in *Proc. of the Fifth Starting AI Researchers' Symposium (STAIRS 2010)*, 2011, pp. 140–150.
- [17] J. Domènech, "All my sokoban collections," <https://sokoban-jd.blogspot.com/p/all-my-sokoban-collections.html>, n.d., accessed: 2026-04-14.
- [18] J. Chu, K. Zheng, and J. E. Fan, "What makes people think a puzzle is fun to solve?" in *Proceedings of the Annual Meeting of the Cognitive Science Society*, vol. 47, 2025.
- [19] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, and T. Stützle, "The irace package: Iterated racing for automatic algorithm configuration," *Operations Research Perspectives*, vol. 3, pp. 43–58, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2214716015300270>